

PROGRAM 1

Develop a c program to implement the Process system calls (fork (), exec(), wait(), create process, terminate process)

```
#include <stdio.h>

#include <stdlib.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <unistd.h>


int main() {

    pid_t child_pid;


    // Fork a child process

    child_pid = fork();


    if (child_pid == -1) {

        // Fork failed

        perror("Fork failed");

        exit(EXIT_FAILURE);

    }


    if (child_pid == 0) {

        // This is the child process


        printf("Child process: PID = %d\n", getpid());


        // Execute a new program in the child process

        execlp("/bin/ls", "ls", "-l", (char *)NULL);


        // If exec fails

        perror("Exec failed");

        exit(EXIT_FAILURE);

    }

}
```

```
} else {  
    // This is the parent process  
  
    printf("Parent process: PID = %d\n", getpid());  
  
    // Wait for the child process to finish  
    wait(NULL);  
  
    printf("Child process terminated\n");  
}  
  
return 0;  
}
```

SAMPLE OUTPUT

Parent process: PID = 12345

Child process: PID = 12346

total 12

-rwxrwxr-x 1 user user 10000 Nov 22 15:00 your_program_name

Child process terminated

PROGRAM 2

Simulate the following CPU scheduling algorithms to find turnaround time and waiting time a) FCFS
b) SJF c) Round Robin d) Priority.

```
#include <stdio.h>
```

```
void findWaitingTime(int processes[], int n, int bt[], int wt[]) {  
    wt[0] = 0;  
  
    for (int i = 1; i < n; i++) {  
        wt[i] = bt[i - 1] + wt[i - 1];  
    }  
}
```

```
void findTurnaroundTime(int processes[], int n, int bt[], int wt[], int tat[]) {  
    for (int i = 0; i < n; i++) {  
        tat[i] = bt[i] + wt[i];  
    }  
}
```

```
void findAverageTime(int processes[], int n, int bt[]) {  
    int wt[n], tat[n];  
  
    findWaitingTime(processes, n, bt, wt);  
    findTurnaroundTime(processes, n, bt, wt, tat);  
  
    double total_wt = 0, total_tat = 0;  
    for (int i = 0; i < n; i++) {  
        total_wt += wt[i];  
        total_tat += tat[i];  
    }  
  
    printf("Average Waiting Time: %.2f\n", total_wt / n);  
    printf("Average Turnaround Time: %.2f\n", total_tat / n);  
}
```

```

void fcfs(int processes[], int n, int bt[]) {
    printf("\nFirst-Come-First-Serve (FCFS):\n");
    findAverageTime(processes, n, bt);
}

```

```

void sjf(int processes[], int n, int bt[]) {
    // Sorting processes based on burst time for SJF

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (bt[j] > bt[j + 1]) {
                // Swap

                int temp = bt[j];
                bt[j] = bt[j + 1];
                bt[j + 1] = temp;

                temp = processes[j];
                processes[j] = processes[j + 1];
                processes[j + 1] = temp;
            }
        }
    }
}

```

```

    printf("\nShortest Job First (SJF):\n");
    findAverageTime(processes, n, bt);
}

```

```

void roundRobin(int processes[], int n, int bt[], int quantum) {
    printf("\nRound Robin (RR) with quantum = %d:\n", quantum);

    int wt[n], tat[n];

    int remaining[n];
    for (int i = 0; i < n; i++) {
        remaining[i] = bt[i];
    }

    int t = 0; // Current time
}

```

```

while (1) {

    int done = 1;

    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            done = 0; // There is a pending process

            if (remaining[i] > quantum) {
                t += quantum;
                remaining[i] -= quantum;
            } else {
                t += remaining[i];
                wt[i] = t - bt[i];
                remaining[i] = 0;
            }
        }
    }

    if (done == 1) {
        break;
    }
}

findTurnaroundTime(processes, n, bt, wt, tat);

double total_wt = 0, total_tat = 0;

for (int i = 0; i < n; i++) {
    total_wt += wt[i];
    total_tat += tat[i];
}

printf("Average Waiting Time: %.2f\n", total_wt / n);
printf("Average Turnaround Time: %.2f\n", total_tat / n);

}

void priority(int processes[], int n, int bt[], int priority[]) {
    // Sorting processes based on priority for Priority Scheduling
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (priority[j] > priority[j + 1]) {
                // Swap
            }
        }
    }
}

```

```
int temp = bt[j];
```

```
bt[j] = bt[j + 1];
```

```
bt[j + 1] = temp;
```

```
temp = processes[j];
```

```
processes[j] = processes[j + 1];
```

```
processes[j + 1] = temp;
```

```
temp = priority[j];
```

```
priority[j] = priority[j + 1];
```

```
priority[j + 1] = temp;
```

```
}
```

```
}
```

```
}
```

```
printf("\nPriority Scheduling:\n");
```

```
findAverageTime(processes, n, bt);
```

```
}
```

```
int main() {
```

```
int processes[] = {1, 2, 3};
```

```
int n = sizeof(processes) / sizeof(processes[0]);
```

```
int burst_time[] = {6, 8, 10};
```

```
int priority[] = {3, 1, 2};
```

```
int quantum = 2;
```

```
fcfs(processes, n, burst_time);
```

```
sjf(processes, n, burst_time);
```

```
roundRobin(processes, n, burst_time, quantum);
```

```
priority(processes, n, burst_time, priority);
```

```
return 0;
```

```
}
```

/*SAMPLE OUTPUT

First-Come-First-Serve (FCFS):

Average Waiting Time: 0.00

Average Turnaround Time: 8.00

Shortest Job First (SJF):

Average Waiting Time: 3.33

Average Turnaround Time: 7.33

Round Robin (RR) with quantum = 2:

Average Waiting Time: 6.00

Average Turnaround Time: 10.00

Priority Scheduling:

Average Waiting Time: 0.67

Average Turnaround Time: 8.67*/

PROGRAM 3

Develop a C program to simulate producer-consumer problem using semaphores.

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

sem_t empty, full;
pthread_mutex_t mutex;

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

void *producer(void *arg) {
    int item;
    for (int i = 0; i < BUFFER_SIZE * 2; i++) {
        item = rand() % 100; // Produce a random item

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        // Critical Section: Add item to the buffer
        buffer[in] = item;
        printf("Produced: %d\n", item);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
}
```

```

        // Simulate some work in between producing items
        usleep(100000);
    }
    pthread_exit(NULL);
}

```

```

void *consumer(void *arg) {
    int item;
    for (int i = 0; i < BUFFER_SIZE * 2; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        // Critical Section: Consume item from the buffer
        item = buffer[out];
        printf("Consumed: %d\n", item);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        // Simulate some work in between consuming items
        usleep(150000);
    }
    pthread_exit(NULL);
}

```

```

int main() {
    pthread_t producerThread, consumerThread;

    // Initialize semaphores and mutex
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

```

```
pthread_mutex_init(&mutex, NULL);

// Create producer and consumer threads
pthread_create(&producerThread, NULL, producer, NULL);
pthread_create(&consumerThread, NULL, consumer, NULL);

// Join threads
pthread_join(producerThread, NULL);
pthread_join(consumerThread, NULL);

// Destroy semaphores and mutex
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0;
}
```

SAMPLE OUTPUT

Produced: 49
Produced: 13
Produced: 64
Produced: 72
Produced: 87
Consumed: 49
Consumed: 13
Produced: 41
Produced: 21
Produced: 95
Produced: 87
Consumed: 64
Produced: 68

Consumed: 72

Consumed: 87

Consumed: 41

Produced: 61

Consumed: 21

Produced: 30

Consumed: 95

Produced: 58

Consumed: 87

Produced: 4

Consumed: 68

Produced: 75

Consumed: 61

Consumed: 30

Produced: 96

Produced: 39

Consumed: 58

Produced: 60

Consumed: 4

Consumed: 75

Consumed: 96

Produced: 48

Produced: 3

Consumed: 39

Produced: 94

Consumed: 60

Consumed: 48

Produced: 82

Consumed: 3

PROGRAM 4

Develop a C program which demonstrates interprocess communication between a reader process and a writer process. Use mkfifo, open, read, write and close APIs in your program.

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define FIFO_NAME "myfifo"

void writerProcess() {
    int fd;

    // Create the FIFO (named pipe)
    mkfifo(FIFO_NAME, 0666);

    // Open the FIFO for writing
    fd = open(FIFO_NAME, O_WRONLY);

    if (fd == -1) {
        perror("Error opening FIFO for writing");
        exit(EXIT_FAILURE);
    }

    // Write data to the FIFO
    char message[] = "Hello, reader!";
    write(fd, message, strlen(message) + 1);

    // Close the FIFO
```

```

    close(fd);
}

void readerProcess() {
    int fd;
    char buffer[100];

    // Open the FIFO for reading
    fd = open(FIFO_NAME, O_RDONLY);

    if (fd == -1) {
        perror("Error opening FIFO for reading");
        exit(EXIT_FAILURE);
    }

    // Read data from the FIFO
    read(fd, buffer, sizeof(buffer));

    // Display the read data
    printf("Message from writer: %s\n", buffer);

    // Close the FIFO
    close(fd);
}

int main() {
    pid_t child_pid;

    // Fork a child process
    child_pid = fork();

    if (child_pid == -1) {

```

```
perror("Fork failed");
exit(EXIT_FAILURE);
}

if (child_pid == 0) {
    // This is the writer process
    writerProcess();
} else {
    // This is the reader process
    readerProcess();
}

return 0;
}
```

SAMPLE OUTPUT

Message from writer: Hello, reader!

PROGRAM 5

Develop a C program to simulate Bankers Algorithm for DeadLock Avoidance.

```
#include <stdio.h>

#define MAX_PROCESSES 5
#define MAX_RESOURCES 3

int available[MAX_RESOURCES];
int maximum[MAX_PROCESSES][MAX_RESOURCES];
int allocation[MAX_PROCESSES][MAX_RESOURCES];
int need[MAX_PROCESSES][MAX_RESOURCES];
int request_resources(int process_id, int request[]);
int release_resources(int process_id, int release[]);
int is_safe_state();

int main() {
    // Initialize available resources
    printf("Enter the number of available resources:\n");
    for (int i = 0; i < MAX_RESOURCES; i++) {
        scanf("%d", &available[i]);
    }

    // Initialize maximum matrix
    printf("Enter the maximum demand matrix:\n");
    for (int i = 0; i < MAX_PROCESSES; i++) {
        for (int j = 0; j < MAX_RESOURCES; j++) {
            scanf("%d", &maximum[i][j]);
        }
    }

    // Initialize allocation matrix
    printf("Enter the allocation matrix:\n");
    for (int i = 0; i < MAX_PROCESSES; i++) {
        for (int j = 0; j < MAX_RESOURCES; j++) {
            scanf("%d", &allocation[i][j]);
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}
```

```

    }
}

// Demonstrate resource request and release
int process_id;
int request[MAX_RESOURCES];
int release[MAX_RESOURCES];

printf("Enter the process ID (0 to %d):\n", MAX_PROCESSES - 1);
scanf("%d", &process_id);

printf("Enter the resource request:\n");
for (int i = 0; i < MAX_RESOURCES; i++) {
    scanf("%d", &request[i]);
}

if (request_resources(process_id, request) == 1) {
    printf("Resource request granted. System is in a safe state.\n");

    printf("Enter the resources to release:\n");
    for (int i = 0; i < MAX_RESOURCES; i++) {
        scanf("%d", &release[i]);
    }

    release_resources(process_id, release);
} else {
    printf("Resource request denied. System would be in an unsafe state.\n");
}

return 0;
}

int request_resources(int process_id, int request[]) {
    // Check if the request is within the need of the process
    for (int i = 0; i < MAX_RESOURCES; i++) {
        if (request[i] > need[process_id][i]) {
            return 0; // Request exceeds need
        }
    }
}

```

```

// Check if the request is within the available resources
for (int i = 0; i < MAX_RESOURCES; i++) {
    if (request[i] > available[i]) {
        return 0; // Insufficient resources
    }
}

// Try to allocate resources temporarily
for (int i = 0; i < MAX_RESOURCES; i++) {
    available[i] -= request[i];
    allocation[process_id][i] += request[i];
    need[process_id][i] -= request[i];
}

// Check if the system is in a safe state
if (is_safe_state()) {
    return 1; // Request granted
} else {
    // Roll back changes if the system would be in an unsafe state
    for (int i = 0; i < MAX_RESOURCES; i++) {
        available[i] += request[i];
        allocation[process_id][i] -= request[i];
        need[process_id][i] += request[i];
    }

    return 0; // Request denied
}
}

int release_resources(int process_id, int release[]) {
    // Release resources

    for (int i = 0; i < MAX_RESOURCES; i++) {
        available[i] += release[i];
        allocation[process_id][i] -= release[i];
        need[process_id][i] += release[i];
    }
}

```

```

// Check if the system is in a safe state
if (is_safe_state()) {
    printf("Resources released. System is in a safe state.\n");
    return 1; // Resources released
} else {
    // Roll back changes if the system would be in an unsafe state
    for (int i = 0; i < MAX_RESOURCES; i++) {
        available[i] -= release[i];
        allocation[process_id][i] += release[i];
        need[process_id][i] -= release[i];
    }

    printf("Resources cannot be released. System would be in an unsafe state.\n");
    return 0; // Resources not released
}
}

```

```

int is_safe_state() {
    int work[MAX_RESOURCES];
    int finish[MAX_PROCESSES];

    // Initialize work and finish arrays
    for (int i = 0; i < MAX_RESOURCES; i++) {
        work[i] = available[i];
    }

    for (int i = 0; i < MAX_PROCESSES; i++) {
        finish[i] = 0;
    }

    // Try to find an unmarked process that needs fewer resources than available
    int found;
    do {
        found = 0;

        for (int i = 0; i < MAX_PROCESSES; i++) {
            if (finish[i] == 0) {
                int j;

```

```

    for (j = 0; j < MAX_RESOURCES; j++) {
        if (need[i][j] > work[j]) {
            break;
        }
    }

    if (j == MAX_RESOURCES) {
        // Process i can finish
        for (int k = 0; k < MAX_RESOURCES; k++) {
            work[k] += allocation[i][k];
        }

        finish[i] = 1;
        found = 1;
    }
}
} while (found);

// Check if all processes are finished

for (int i = 0; i < MAX_PROCESSES; i++) {
    if (finish[i] == 0) {
        return 0; // System is in an unsafe state
    }
}

return 1; // System is in a safe state
}

```

SAMPLE OUTPUT

Enter the number of available resources:

3 3 2

Enter the maximum demand matrix:

7 5 3

3 2 2

9 0 2

2 2 2

4 3 3

Enter the allocation matrix:

0 1 0

2 0 0

3 0 2

2 1 1

0 0 2

Enter the process ID (0 to 4):

1

Enter the resource request:

0 1 0

Resource request granted. System is in a safe state.

Enter the resources to release:

0 1 0

Resources released. System is in a safe state.

PROGRAM 6

Develop a C program to simulate the following contiguous memory allocation Techniques: a) Worst fit b) Best fit c) First fit

```
#include <stdio.h>
```

```
#define MEMORY_SIZE 100
```

```
struct MemoryBlock {  
    int process_id;  
    int size;  
    int start_address;  
};
```

```
void initializeMemory(struct MemoryBlock memory[], int size) {  
    for (int i = 0; i < size; i++) {  
        memory[i].process_id = -1;  
        memory[i].size = 0;  
        memory[i].start_address = -1;  
    }  
}
```

```
void displayMemory(struct MemoryBlock memory[], int size) {  
    printf("Memory State:\n");  
    for (int i = 0; i < size; i++) {  
        if (memory[i].process_id != -1) {  
            printf("Block %d: Process %d (Size: %d)\n", i, memory[i].process_id, memory[i].size);  
        } else {  
            printf("Block %d: Free (Size: %d)\n", i, memory[i].size);  
        }  
    }  
    printf("\n");  
}
```

```

void worstFit(struct MemoryBlock memory[], int num_processes, int process_sizes[]) {
    for (int i = 0; i < num_processes; i++) {
        int largest_hole_index = -1;
        int largest_hole_size = 0;

        for (int j = 0; j < MEMORY_SIZE; j++) {
            if (memory[j].process_id == -1 && memory[j].size >= process_sizes[i]) {
                if (largest_hole_index == -1 || memory[j].size > largest_hole_size) {
                    largest_hole_index = j;
                    largest_hole_size = memory[j].size;
                }
            }
        }

        if (largest_hole_index != -1) {
            // Allocate memory
            memory[largest_hole_index].process_id = i + 1;
            memory[largest_hole_index].size = process_sizes[i];
            memory[largest_hole_index].start_address = largest_hole_index;
        } else {
            printf("Process %d (Size: %d) cannot be allocated. No suitable block found.\n", i + 1,
                process_sizes[i]);
        }
    }
}

```

```

void bestFit(struct MemoryBlock memory[], int num_processes, int process_sizes[]) {
    for (int i = 0; i < num_processes; i++) {
        int best_fit_index = -1;
        int best_fit_size = MEMORY_SIZE + 1;

        for (int j = 0; j < MEMORY_SIZE; j++) {

```

```

        if (memory[j].process_id == -1 && memory[j].size >= process_sizes[i]) {
            if (memory[j].size < best_fit_size) {
                best_fit_index = j;
                best_fit_size = memory[j].size;
            }
        }
    }

    if (best_fit_index != -1) {
        // Allocate memory
        memory[best_fit_index].process_id = i + 1;
        memory[best_fit_index].size = process_sizes[i];
        memory[best_fit_index].start_address = best_fit_index;
    } else {
        printf("Process %d (Size: %d) cannot be allocated. No suitable block found.\n", i + 1,
process_sizes[i]);
    }
}
}

```

```

void firstFit(struct MemoryBlock memory[], int num_processes, int process_sizes[]) {
    for (int i = 0; i < num_processes; i++) {
        int first_fit_index = -1;

        for (int j = 0; j < MEMORY_SIZE; j++) {
            if (memory[j].process_id == -1 && memory[j].size >= process_sizes[i]) {
                first_fit_index = j;
                break;
            }
        }

        if (first_fit_index != -1) {
            // Allocate memory

            memory[first_fit_index].process_id = i + 1;
            memory[first_fit_index].size = process_sizes[i];
            memory[first_fit_index].start_address = first_fit_index;

```

```

    } else {

        printf("Process %d (Size: %d) cannot be allocated. No suitable block found.\n", i + 1,
process_sizes[i]);

    }

}

}

int main() {

    int num_processes;

    printf("Enter the number of processes: ");

    scanf("%d", &num_processes);

    int process_sizes[num_processes];

    printf("Enter the size of each process:\n");

    for (int i = 0; i < num_processes; i++) {

        scanf("%d", &process_sizes[i]);

    }

    struct MemoryBlock memory[MEMORY_SIZE];

    initializeMemory(memory, MEMORY_SIZE);

    printf("\nWorst Fit Allocation:\n");

    worstFit(memory, num_processes, process_sizes);

    displayMemory(memory, MEMORY_SIZE);

    initializeMemory(memory, MEMORY_SIZE);

    printf("Best Fit Allocation:\n");

    bestFit(memory, num_processes, process_sizes);

    displayMemory(memory, MEMORY_SIZE);

    initializeMemory(memory, MEMORY_SIZE);

    printf("First Fit Allocation:\n");

    firstFit(memory, num_processes, process_sizes);

    displayMemory(memory, MEMORY_SIZE);

    return 0;

}

```

SAMPLE OUTPUT

Enter the number of processes: 4

Enter the size of each process:

10 20 30 40

Worst Fit Allocation:

Process 1 (Size: 10) allocated to Block 0 (Size: 10)

Process 2 (Size: 20) allocated to Block 10 (Size: 20)

Process 3 (Size: 30) cannot be allocated. No suitable block found.

Process 4 (Size: 40) allocated to Block 30 (Size: 40)

Memory State:

Block 0: Process 1 (Size: 10)

Block 10: Process 2 (Size: 20)

Block 20: Free (Size: 20)

Block 30: Process 4 (Size: 40)

Block 70: Free (Size: 30)

Best Fit Allocation:

Process 1 (Size: 10) allocated to Block 0 (Size: 10)

Process 2 (Size: 20) allocated to Block 10 (Size: 20)

Process 3 (Size: 30) cannot be allocated. No suitable block was found.

Process 4 (Size: 40) allocated to Block 30 (Size: 40)

Memory State:

Block 0: Process 1 (Size: 10)

Block 10: Process 2 (Size: 20)

Block 20: Free (Size: 20)

Block 30: Process 4 (Size: 40)

Block 70: Free (Size: 30)

First Fit Allocation:

Process 1 (Size: 10) allocated to Block 0 (Size: 10)

Process 2 (Size: 20) allocated to Block 10 (Size: 20)

Process 3 (Size: 30) allocated to Block 30 (Size: 30)

Process 4 (Size: 40) cannot be allocated. No suitable block was found.

Memory State:

Block 0: Process 1 (Size: 10)

Block 10: Process 2 (Size: 20)

Block 30: Process 3 (Size: 30)

Block 60: Free (Size: 40)

Block 100: Free (Size: 30)

PROGRAM 7

Develop a C program to simulate page replacement algorithms: a) FIFO b) LRU

```
#include <stdio.h>

#include <stdlib.h>

#define FRAME_SIZE 3
#define REFERENCE_STRING_SIZE 10

void fifo(int reference_string[], int size) {
    int frame[FRAME_SIZE];
    int frame_index = 0;
    int page_faults = 0;
    printf("\nFIFO Page Replacement Algorithm:\n");

    for (int i = 0; i < size; i++) {
        int page = reference_string[i];
        int page_found = 0;
        // Check if the page is already in the frame
        for (int j = 0; j < FRAME_SIZE; j++) {
            if (frame[j] == page) {
                page_found = 1;
                break;
            }
        }
        if (!page_found) {
            // Page fault
            printf("Page %d caused a page fault.\n", page);
            if (frame_index < FRAME_SIZE) {
                // There is space in the frame
                frame[frame_index++] = page;
            } else {
                // Replace the oldest page in the frame
                for (int j = 0; j < FRAME_SIZE - 1; j++) {
                    frame[j] = frame[j + 1];
                }
                frame[FRAME_SIZE - 1] = page;
            }
        }
    }
}
```

```

        page_faults++;
    }

    // Display the current state of the frame
    printf("Frame: ");
    for (int j = 0; j < frame_index; j++) {
        printf("%d ", frame[j]);
    }
    printf("\n");
}
printf("Total Page Faults: %d\n", page_faults);
}

void lru(int reference_string[], int size) {
    int frame[FRAME_SIZE];
    int time_used[FRAME_SIZE];
    int page_faults = 0;
    printf("\nLRU Page Replacement Algorithm:\n");
    for (int i = 0; i < size; i++) {
        int page = reference_string[i];
        int page_found = 0;
        // Check if the page is already in the frame
        for (int j = 0; j < FRAME_SIZE; j++) {
            if (frame[j] == page) {
                page_found = 1;

                time_used[j] = i; // Update time used
                break;
            }
        }
        if (!page_found) {
            // Page fault
            printf("Page %d caused a page fault.\n", page);

            if (i < FRAME_SIZE) {
                // There is space in the frame
                frame[i] = page;
                time_used[i] = i;
            } else {

```

```

// Find the least recently used page in the frame
int min_time_index = 0;
for (int j = 1; j < FRAME_SIZE; j++) {
    if (time_used[j] < time_used[min_time_index]) {
        min_time_index = j;
    }
}

// Replace the least recently used page
frame[min_time_index] = page;
time_used[min_time_index] = i;
}

page_faults++;
}

// Display the current state of the frame
printf("Frame: ");
for (int j = 0; j < FRAME_SIZE; j++) { printf("%d ", frame[j]);
}
printf("\n");
}
printf("Total Page Faults: %d\n", page_faults);
}

int main() {
    int reference_string[REFERENCE_STRING_SIZE] = {1, 2, 3, 4, 2, 1, 5, 6, 2, 1};
    fifo(reference_string, REFERENCE_STRING_SIZE);

    lru(reference_string, REFERENCE_STRING_SIZE);
    return 0;
}

```

SAMPLE OUTPUT

Step	Page	FIFO Algorithm (Frames)	LRU Algorithm (Frames)	Status
1	1	[1]	[1]	Page Fault
2	2	[1, 2]	[1, 2]	Page Fault
3	3	[1, 2, 3]	[1, 2, 3]	Page Fault
4	4	[2, 3, 4]	[2, 3, 4]	Page Fault
5	2	[3, 4, 2]	[3, 4, 2]	Page Fault
6	1	[4, 2, 1]	[4, 2, 1]	Page Fault
7	5	[2, 1, 5]	[2, 1, 5]	Page Fault
8	6	[1, 5, 6]	[1, 5, 6]	Page Fault
9	2	[5, 6, 2]	[5, 6, 2]	Page Fault
10	1	[6, 2, 1]	[6, 2, 1]	Page Fault
Total		10 Faults	10 Faults	

PROGRAM 8

Simulate following File Organization Techniques a) Single level directory b) Two level directory

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure for a file entry
struct File {
    char filename[30];
    int size;
};

// Structure for a directory entry
struct Directory {
    char dirname[30];
    int num_files;
    struct File files[10];
};

// Function to display the contents of a single-level directory
void displaySingleLevelDirectory(struct Directory directory) {
    printf("\nSingle-Level Directory:\n");
    printf("Directory Name: %s\n", directory.dirname);
    printf("Number of Files: %d\n", directory.num_files);
    printf("Files:\n");

    for (int i = 0; i < directory.num_files; i++) {
        printf("File Name: %s, Size: %d\n", directory.files[i].filename, directory.files[i].size);
    }
}

// Function to display the contents of a two-level directory
void displayTwoLevelDirectory(struct Directory directories[], int num_directories) {
    printf("\nTwo-Level Directory:\n");
    for (int i = 0; i < num_directories; i++) {
        printf("Directory Name: %s\n", directories[i].dirname);
        printf("Number of Files: %d\n", directories[i].num_files);
        printf("Files:\n");

        for (int j = 0; j < directories[i].num_files; j++) {
```

```

        printf("File Name: %s, Size: %d\n", directories[i].files[j].filename, directories[i].files[j].size);
    }

    printf("\n");
}

}

int main() {
    // Single-Level Directory
    struct Directory singleLevelDirectory;
    strcpy(singleLevelDirectory.dirname, "root");
    singleLevelDirectory.num_files = 3;

    strcpy(singleLevelDirectory.files[0].filename, "file1.txt");
    singleLevelDirectory.files[0].size = 100;

    strcpy(singleLevelDirectory.files[1].filename, "file2.txt");
    singleLevelDirectory.files[1].size = 150;

    strcpy(singleLevelDirectory.files[2].filename, "file3.txt");
    singleLevelDirectory.files[2].size = 200;

    displaySingleLevelDirectory(singleLevelDirectory);

    // Two-Level Directory
    struct Directory twoLevelDirectories[2];
    twoLevelDirectories[0].num_files = 2;
    strcpy(twoLevelDirectories[0].dirname, "user1");

    strcpy(twoLevelDirectories[0].files[0].filename, "file1.txt");
    twoLevelDirectories[0].files[0].size = 120;

    strcpy(twoLevelDirectories[0].files[1].filename, "file2.txt");
    twoLevelDirectories[0].files[1].size = 180;

    twoLevelDirectories[1].num_files = 1;
    strcpy(twoLevelDirectories[1].dirname, "user2");

    strcpy(twoLevelDirectories[1].files[0].filename, "file3.txt");
    twoLevelDirectories[1].files[0].size = 220;

```

```
displayTwoLevelDirectory(twoLevelDirectories, 2);  
  
return 0;  
}
```

SAMPLE OUTPUT

Single-Level Directory:

Directory Name: root

Number of Files: 3

Files:

File Name: file1.txt, Size: 100

File Name: file2.txt, Size: 150

File Name: file3.txt, Size: 200

Two-Level Directory:

Directory Name: user1

Number of Files: 2

Files:

File Name: file1.txt, Size: 120

File Name: file2.txt, Size: 180

Directory Name: user2

Number of Files: 1

Files:

File Name: file3.txt, Size: 220

Develop a C program to simulate the Linked file allocation strategies.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// Structure for a file block
struct FileBlock {
    int block_number;
    struct FileBlock* next_block;
};
// Structure for a file entry
struct File {
    char filename[30];
    struct FileBlock* first_block;
};
// Function to create a new file block
struct FileBlock* createFileBlock(int block_number) {
    struct FileBlock* new_block = (struct FileBlock*)malloc(sizeof(struct FileBlock));
    new_block->block_number = block_number;
    new_block->next_block = NULL;
    return new_block;
}
// Function to display the blocks of a file
void displayFileBlocks(struct File* file) {
    printf("File Name: %s\n", file->filename);
    printf("File Blocks: ");
    struct FileBlock* current_block = file->first_block;
    while (current_block != NULL) {
        printf("%d ", current_block->block_number);
        current_block = current_block->next_block;
    }
    printf("\n\n");
}
```

```

// Function to add a new block to a file

void addFileBlock(struct File* file, int block_number) {

    struct FileBlock* new_block = createFileBlock(block_number);

    if (file->first_block == NULL) {
        file->first_block = new_block;

    } else {

        struct FileBlock* current_block = file->first_block;

        while (current_block->next_block != NULL) {
            current_block = current_block->next_block;
        }

        current_block->next_block = new_block;
    }
}

int main() {

    // Create files with linked blocks
    struct File file1, file2, file3;
    strcpy(file1.filename, "file1.txt");
    file1.first_block = NULL;
    addFileBlock(&file1, 1);
    addFileBlock(&file1, 2);
    addFileBlock(&file1, 3);

    strcpy(file2.filename, "file2.txt");
    file2.first_block = NULL;
    addFileBlock(&file2, 4);
    addFileBlock(&file2, 5);

    strcpy(file3.filename, "file3.txt");
    file3.first_block = NULL;
    addFileBlock(&file3, 6);

    // Display file blocks
    displayFileBlocks(&file1);
    displayFileBlocks(&file2);
    displayFileBlocks(&file3);

    // Free allocated memory for file blocks

```

```

struct FileBlock* current_block;
while (file1.first_block != NULL) {
    current_block = file1.first_block;
    file1.first_block = file1.first_block->next_block;
    free(current_block);
}

while (file2.first_block != NULL) {
    current_block = file2.first_block;
    file2.first_block = file2.first_block->next_block;
    free(current_block);
}

while (file3.first_block != NULL) {
    current_block = file3.first_block;
    file3.first_block = file3.first_block->next_block;
    free(current_block);
}

return 0;
}

```

SAMPLE OUTPUT

File Name: file1.txt

File Blocks: 1 2 3

File Name: file2.txt

File Blocks: 4 5

File Name: file3.txt

File Blocks: 6

Allocated memory for file blocks has been freed.

Develop a C program to simulate SCAN disk scheduling algorithm.

```
#include <stdio.h>

#include <stdlib.h>

// Function to sort the array of disk requests
void sortRequests(int requests[], int num_requests) {
    for (int i = 0; i < num_requests - 1; i++) {
        for (int j = 0; j < num_requests - i - 1; j++) {
            if (requests[j] > requests[j + 1]) {
                // Swap requests[j] and requests[j+1]
                int temp = requests[j];
                requests[j] = requests[j + 1];
                requests[j + 1] = temp;
            }
        }
    }
}

// Function to simulate the SCAN disk scheduling algorithm
void scan(int requests[], int num_requests, int start_position, int direction) {
    // Sort the requests
    sortRequests(requests, num_requests);

    int head_position = start_position; int total_head_movement = 0;

    // Find the index of the head position in the sorted requests
    int head_index;
    for (head_index = 0; head_index < num_requests; head_index++) {
        if (requests[head_index] >= head_position) {
            break;
        }
    }

    printf("Sequence of disk requests using SCAN algorithm:\n");

    // Scan towards the end
    for (int i = head_index; i < num_requests; i++) {
        printf("%d ", requests[i]);

        total_head_movement += abs(requests[i] - head_position);
    }
}
```

```

        head_position = requests[i];
    }

    // If direction is left, move to the end of the disk
    if (direction == -1) {
        printf("0 ");
        total_head_movement += head_position;
        head_position = 0;
    }

    // Scan towards the beginning
    for (int i = head_index - 1; i >= 0; i--) {
        printf("%d ", requests[i]);
        total_head_movement += abs(requests[i] - head_position);
        head_position = requests[i];
    }

    printf("\nTotal head movement: %d\n", total_head_movement);
}

int main() {
    int num_requests;

    printf("Enter the number of disk requests: ");
    scanf("%d", &num_requests);

    int requests[num_requests];

    printf("Enter the disk requests:\n");
    for (int i = 0; i < num_requests; i++) {
        scanf("%d", &requests[i]);
    }

    int start_position;

    printf("Enter the starting head position: ");
    scanf("%d", &start_position);

    int direction;

    printf("Enter the direction (1 for right, -1 for left): ");
    scanf("%d", &direction);

```

```
scan(requests, num_requests, start_position, direction);
```

```
return 0;
```

```
}
```

SAMPLE OUTPUT

Enter the number of disk requests: 8

Enter the disk requests:

98 183 37 122 14 124 65 67

Enter the starting head position: 53

Enter the direction (1 for right, -1 for left): 1

Sequence of disk requests using SCAN algorithm:

65 67 98 122 124 124 183 183 37 14

Total head movement: 236